Performance Analysis of Apache ZooKeeper's Write Pipeline

Software Performance Engineering — Project Stage II

Gil Jorge (110062), João Palma (109242), and Gabriel Petti Tavares (120691)

Instituto Superior Técnico, Universidade de Lisboa — Group 18
gil.jorge@tecnico.ulisboa.pt, joaofontepalma@tecnico.ulisboa.pt,
gabriel.p.tavares@tecnico.ulisboa.pt

Abstract. ZooKeeper answers reads at any server but sends every write through one leader and a quorum. We study the cost of that write pipeline on Google Compute Engine with a 2^{6-1} factorial experiment (two replicates), one replicated Universal Scalability Law curve and five eBPF probes. The write ratio explains 60% of the variation of $\log_{10}$ throughput and the number of clients 94% of the variation of $\log_{10}$ write tail latency. Turning `fsync` off helps at light load (+39%) and hurts at saturation (−17%). The curve needs only a contention term ($\sigma = 0.039$). The utilization law and a CPU profile of the leader agree with the leader's two vCPUs being the bottleneck; the log disk is not. We also report a client-side logging artifact that had distorted the Stage I fits.

Git Commit Identifier: `9a52eabcfe0019b8ab4ac96fb7bb2195694d70a0`

1 Introduction

Apache ZooKeeper is a coordination service used by Kafka, HBase and Hadoop for leader election, configuration and locks [1, 2]. Its replication protocol, Zab [3], makes reads and writes behave very differently: a read is answered by the server the client is connected to, while a write goes to the leader, is logged and acknowledged by a majority and only then committed. Reads should therefore scale with the number of servers and writes should not, and the write pipeline is where we look for the limits of scalability.

Stage I measured one laptop with Docker and fitted the Universal Scalability Law (USL) [4, 5]. In Stage II we (1) found and removed a client-side artifact in the Stage I data (Section 3); (2) ran a fractional factorial experiment on dedicated virtual machines to rank the factors that drive throughput and write tail latency (Sections 5–6.1); (3) fitted one replicated USL curve and recorded ZooKeeper's per-stage timers at every load level (Section 6.2); and (4) observed the leader with eBPF (Section 6.3).

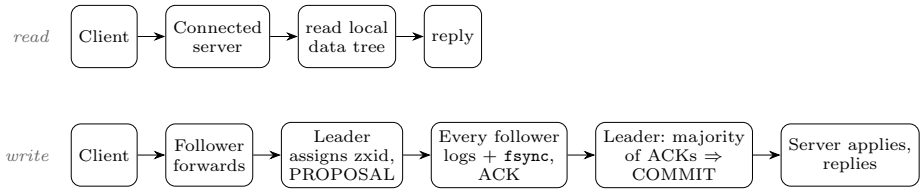


Fig. 1. Read and write paths in ZooKeeper.

2 System Description

ZooKeeper keeps a tree of small data nodes (znodes) in memory. Clients read and write whole znodes (at most 1 MB), can set one-time *watches* that notify them of a change, and can create *ephemeral* znodes that disappear when their session ends; locks, barriers and leader election are built from these. Each client holds a session with one server, kept alive by heartbeats. Every change is appended to a transaction log, and the in-memory tree is written periodically to a snapshot. $N = 2f + 1$ voting servers elect a *leader*, tolerate f failures and stop accepting writes when no majority is reachable [3]; *observers* receive the committed writes but do not vote. Writes are linearizable; reads are sequentially consistent per session and may be slightly stale, which is what allows a follower to answer them alone [1]. Figure 1 shows both paths.

Inside the leader a write passes a chain of processors. Request preparation (one thread) validates it and assigns the zxid; the log-sync stage (one thread) appends it to the log and calls `fsync`, writing every request that is queued at that moment in one batch (group commit); the commit stage applies it once a majority has acknowledged. The followers run the same log-sync stage before they acknowledge. The commit stage is shared with reads: a read on a server waits while a write of the same session is pending there. A read thus costs one hop and no coordination; a write costs a forward, a proposal to every follower, a log write and `fsync` on each, acknowledgments and a commit.

3 Stage I

In Stage I the same configuration (3 servers, 20 clients, 30% writes) gave 6,364 ops/s in the sweep and 14,832 ops/s when we repeated it later. The jar used for the sweep bundled Logback, whose default configuration logs the ZooKeeper client at DEBUG level: it printed about 145,000 lines in a 15 s run, one per operation, and all client threads waited on the console. A one-line Logback configuration fixes it, and the runner now aborts any run in which the client prints more than a few lines. The Stage I fits had given the same contention coefficient (≈ 0.23) for 1, 3 and 5 servers, which we read as server-side contention unrelated to Zab; a bottleneck in the client also produces a coefficient that does not depend on the number of servers. With the quiet client the coefficient was 0.025, 0.088 and 0.112 for 1, 3 and 5 servers. Repetitions of one variant on the laptop also

differed by up to a factor of two, so Stage II uses separate machines for client and servers, replicates and a randomized order. The Stage I experiments that were not repeated on the new testbed (YCSB, leader failover, injected network delay) are summarized in Appendix A and are not used for any conclusion of this report.

4 Testbed and Methodology

Deployment. All Stage II runs use Google Compute Engine in one zone (europe-west1-b). The load generator has its own n2-standard-8 machine (8 vCPUs, 32 GB); every ZooKeeper server has an n2-standard-2 (2 vCPUs, 8 GB). All run Debian 12 (kernel 6.1) and OpenJDK 17. ZooKeeper 3.9.3 runs natively with a 1 GB heap. Each server has two extra 20 GB disks, pd-balanced and pd-ssd, formatted with ext4; the transaction log goes on one of them according to the factor *log disk*, and snapshots stay on the boot disk. Before each block of runs the servers are stopped, their data is erased and they are restarted with the new configuration; the runner then checks on every server that the `forceSync` flag and the log directory of the running process are the intended ones.

Load generator. A Java client opens one ZooKeeper session per simulated client, spread over all servers by the connect string. Each client is a closed loop of synchronous calls without think time: it picks one of 200 znodes uniformly and sends a `setData` with probability equal to the write ratio, otherwise a `getData`. All sessions connect before a barrier releases them. Operations that start in the first 5 s are executed but not recorded, so that JIT compilation and cache warm-up fall outside the measurement, and every series starts with a discarded run. A factorial run then measures 20 s and a USL run 30 s. The slowest factorial run still completed more than 30,000 writes, so its 99th percentile rests on over 300 samples. Percentiles are exact. The client's CPU never exceeded 47%.

ZooKeeper's per-stage timers. The `mntr` command reports a count and a sum per stage (queue times, `fsync` time, batch size, time to quorum acknowledgment, commit to applied). Snapshots of every server before and after a run give the mean of that run as the ratio of the differences. The `fsync` timer truncates each call to whole milliseconds, so the `fsync` is also measured with eBPF.

Threats to validity. The virtual machines share physical hosts with other tenants. Replicates and the random order spread that noise over all treatments: two replicates of the same factorial run differ by 11% at the median and by up to 48%. Sessions are placed on a random server, so the share of sessions on the leader varies between runs, most of all with 10 clients; we did not record it. One zone means short round trips between servers. `forceSync=no` is unsafe in production and serves only as a probe.

Table 1. Fixed parameters. Defaults are those of ZooKeeper 3.9 [6].

Parameter	Value
System	
Server machines	n2-standard-2: 2 vCPUs (N2 series, Intel Xeon), 8 GB, Debian 12, kernel 6.1
Network	one VPC subnet in one zone; up to 10 Gbps egress per VM (GCE default for this type)
Server JVM; versions	JDK 17, 1 GB heap, G1; ZooKeeper 3.9.3, client library 3.9.0
Observers; `leaderServes`; snapshots	0; yes (default); boot disk (pd-standard)
`snapCount`, `preAllocSize`, `maxBatchSize`, `flushDelay`	100,000, 64 MB, 1000, 0 (defaults)
`globalOutstandingLimit`; `maxClientCnxns`; `tickTime`	1000 (default); no limit; 2000 ms
Workload	
Operations; key space	`getData/setData` only; 200 znodes, uniform, new pool per run
Client model; session placement	closed loop, synchronous, no think time; random server
Load generator	n2-standard-8 (8 vCPUs, 32 GB), one JVM, one thread per client
Run length	5 s warm-up + 20 s (factorial) or 30 s (USL) measured

5 Experimental Design

Parameters and metrics. Table 1 lists the fixed parameters and Table 2 the factors. The primary response is throughput (successful operations per second), the quantity the USL models. The secondary response is the 99th-percentile write latency, because a coordination service sits on the critical path of other systems. We do not use the mean latency as a separate response: in a closed loop without think time Little's law gives the mean response time $R = N/X$, so it carries no information beyond throughput. Read latency is recorded and reported, but reads do not enter the write pipeline. Failed operations (there were none) and the client's CPU are recorded as validity checks.

Factors and levels. Each factor tests one guess about where a write spends its time (Table 2). We chose 10 and 160 clients from the Stage I curve, where 10 was below the knee. On the new testbed the throughput per client at 10 clients is already about 70% of that of a single client (Section 6.2), so the low level is light load and the high level is deep in saturation. Hunt et al. give 2:1 to 100:1 read-to-write ratios as ZooKeeper's target workloads [1]; a write ratio of 0.1 lies inside that range and 0.5 loads the write path beyond it. Three and five servers are the usual sizes. The payloads bracket small metadata and configuration blobs. Persistent Disk performance depends on disk and machine size, so the two disk types are levels of this testbed only.

Design. We use the half fraction 2^{6-1} with $F = ABCDE$ (defining relation $I = ABCDEF$) [7]. It has resolution VI: main effects are aliased with five-factor interactions and two-factor interactions with four-factor ones, so all six main effects and fifteen two-factor interactions are estimated from 32 runs per replicate if interactions of order four and higher are negligible. A quarter fraction would have resolution IV and alias two-factor interactions in pairs, which is where

Table 2. Factors and levels. "Hard": changing it needs a rebuild of the servers.

Factor	−1	+1	Hard	Why
A number of servers	3	5	yes	quorum size: proposals and ACKs per write
B write ratio	0.1	0.5		share of requests that take the write path
C clients	10	160		light load vs well into saturation
D forceSync	yes	no	yes	removes the fsync from the write path
E payload (bytes)	128	4096		bytes logged, replicated and synced per write
F log disk	pd-balanced	pd-ssd	yes	device under the fsync

Table 3. Sign table of the 2^{6-1} design with the mean of $\log_{10}$ throughput over the two replicates; the last row is the effect q_j of each factor and the grand mean q_0.

Run	A	B	C	D	E	F	$\overline{\log_{10} y}$	Run	A	B	C	D	E	F	$\overline{\log_{10} y}$
1	−	−	−	−	−	−	4.409	17	−	−	−	−	+	+	4.226
2	+	−	−	−	−	+	4.402	18	+	−	−	−	+	−	4.227
3	−	+	−	−	−	+	3.944	19	−	+	−	−	+	−	3.752
4	+	+	−	−	−	−	3.986	20	+	+	−	−	+	+	3.796
5	−	−	+	−	−	+	4.723	21	−	−	+	−	+	−	4.656
6	+	−	+	−	−	−	4.902	22	+	−	+	−	+	+	4.731
7	−	+	+	−	−	−	4.300	23	−	+	+	−	+	+	4.183
8	+	+	+	−	−	+	4.306	24	+	+	+	−	+	−	4.215
9	−	−	−	+	−	+	4.472	25	−	−	−	+	+	−	4.343
10	+	−	−	+	−	−	4.511	26	+	−	−	+	+	+	4.415
11	−	+	−	+	−	−	4.093	27	−	+	−	+	+	+	3.980
12	+	+	−	+	−	+	4.077	28	+	+	−	+	+	−	3.984
13	−	−	+	+	−	−	4.684	29	−	−	+	+	+	+	4.575
14	+	−	+	+	−	+	4.775	30	+	−	+	+	+	−	4.597
15	−	+	+	+	−	+	4.106	31	−	+	+	+	+	−	4.082
16	+	+	+	+	−	−	4.360	32	+	+	+	+	+	+	4.177
q	+0.029	-0.228	+0.149	+0.015	-0.066	-0.007	4.312								

hypotheses H2–H4 sit. Table 3 is the sign table: the full 2^5 table of A–E with F as the product of the other five columns. Two replicates give 64 runs and 32 degrees of freedom for error, which is what the credit allowed. With the measured error this gives intervals of ± 0.015 on each $\log_{10}$ effect, so a change of about 7% between levels is detectable.

Restricted randomization. A, D and F need a rebuild of the servers. Runs sharing their levels form a block (eight blocks of four runs per replicate, 16 rebuilds in total). The block order is randomized in each replicate, and so is the run order inside each block; the plan was generated with a fixed seed before the first run. This is a split-plot structure in which the block is the experimental unit of A, D and F. All effects are tested against the pooled run error, and A, D and F also against a whole-plot error estimated from the 16 block means (replicate $\times$ treatment, $(2-1)(8-1) = 7$ degrees of freedom).

Analysis. With s_{ij} the sign of run i for effect j, $\bar{y}_i$ the mean of the $r = 2$ replicates and $n = 32$, $q_j = \frac{1}{n}\sum_i s_{ij}\bar{y}_i$, $SS_j = nrq_j^2$, $s_e^2 = \sum_{i,k}(y_{ik} - \bar{y}_i)^2/(n(r-1))$ and the interval is $q_j \pm t_{[0.975;\, n(r-1)]}\, s_e/\sqrt{nr}$; and effect j explains $100\, SS_j/SST$ percent of the variation. With 31 effects tested at 95% about two false positives are expected, so effects are ranked by their share of the variation. When $y_{\max}/y_{\min} >$

Table 4. Largest effects ($\log_{10}$ model): q, the factor ($\times$) from the low to the high level, the 95% interval of q and the share of the variation (%). †: interval includes zero.

	throughput (ops/s), $\log_{10}$			write p99 (ms), $\log_{10}$		
Effect	q ($\times$)	95% CI	%	q ($\times$)	95% CI	%
C	0.149 ($\times$1.98)	[0.133, 0.164]	25.6	0.534 ($\times$11.68)	[0.511, 0.556]	93.7
B	-0.228 ($\times$0.35)	[-0.244, -0.213]	60.3	0.041 ($\times$1.21)	[0.018, 0.064]	0.6
E	-0.066 ($\times$0.74)	[-0.081, -0.051]	5.0	0.061 ($\times$1.32)	[0.038, 0.084]	1.2
C×D	-0.056 ($\times$0.77)	[-0.071, -0.041]	3.6	0.059 ($\times$1.31)	[0.036, 0.081]	1.1
A	0.029 ($\times$1.14)	[0.014, 0.044]	1.0	0.021 ($\times$1.10)†	[-0.001, 0.044]	0.2
D	0.015 ($\times$1.07)†	[-0.001, 0.030]	0.3	-0.042 ($\times$0.82)	[-0.065, -0.019]	0.6
E×F	0.021 ($\times$1.10)	[0.006, 0.036]	0.5	-0.004 ($\times$0.98)†	[-0.027, 0.018]	0.0
A×B	0.000 ($\times$1.00)†	[-0.016, 0.015]	0.0	-0.034 ($\times$0.85)	[-0.057, -0.012]	0.4
A×C	0.018 ($\times$1.09)	[0.003, 0.033]	0.4	0.010 ($\times$1.05)†	[-0.013, 0.033]	0.0
B×C	-0.016 ($\times$0.93)	[-0.032, -0.001]	0.3	-0.020 ($\times$0.91)†	[-0.043, 0.003]	0.1
B×E	0.003 ($\times$1.01)†	[-0.012, 0.019]	0.0	-0.022 ($\times$0.90)†	[-0.045, 0.000]	0.2

10 we analyse $\log_{10} y$; residual plots are in Appendix D. The analysis code was first run on synthetic data with known effects.

Hypotheses. From the Stage I results we expected: **H1**: turning `fsync` off (D) raises throughput and lowers the write p99. **H2**: that benefit grows with the write ratio (B×D), since reads never touch the log. **H3**: a faster log disk (F) helps only while `fsync` is on (D×F). **H4**: five servers (A) lower throughput and raise the write p99, mostly through writes (A×B). **H5**: the number of clients (C) has the largest main effect on throughput.

6 Results

6.1 Factorial experiment

All 64 runs completed without errors. Throughput ranged from 5,335 to 81,088 ops/s and the write p99 from 1.2 to 61 ms. Both ratios exceed 10, and the residuals of the additive model widen with the predicted value, so both responses are analysed on $\log_{10}$, where the residuals look homogeneous and roughly normal (Fig. 6). Effects are then multiplicative factors (Table 4); all shares of variation below are of the $\log_{10}$ response. The error explains 2.1% (throughput) and 1.3% (p99), small next to factors that change throughput up to fifteen-fold even though replicates differ by about 11%.

Throughput. The write ratio (B) has the largest effect: going from 10% to 50% writes divides throughput by 2.9 (60% of the variation). If a read costs the servers r and a write w, a throughput ratio q gives $(0.9r+0.1w)/(0.5r+0.5w) = q$. This holds only when server capacity sets throughput, that is, at 160 clients, where $q = 0.32$ and $w \approx 12r$: a write costs about twelve reads. The number of clients (C) comes second ($\times$1.98, 26%), and 4 KiB payloads (E) cost 26% of throughput (5%).

The largest interaction is clients × `forceSync` (C×D, 3.6%, Fig. 2). With 10 clients, turning `fsync` off raises throughput from 12,381 to 17,150 ops/s (geometric means, +39%); with 160 clients it lowers it from 31,776 to 26,270 ops/s (−17%). The reversal holds in every cell: at 160 clients the ratio without and with `fsync` is between 0.79 and 0.87 at all levels of the other factors. At light load each write waits for its own `fsync`, so removing it shortens every request. At saturation group commit already spreads one `fsync` over many requests (Section 6.2), and without it the log-sync stage flushes almost every request on its own (1.9 per flush against 16–17, Table 6). The thread profile shows where that costs: the leader's replication threads use 0.67 cores without `fsync` against 0.55–0.59 with it, at similar throughput, so more, smaller rounds to the followers take CPU that a saturated leader no longer has.

Five servers raise throughput by 14% (×1.14, 1% of the variation), significantly against both errors (×1.08–1.21 in the whole-plot test). The gain depends on the load (A×C): ×1.05 at 10 clients and ×1.24 at 160. It is the same at both write ratios (A×B = ×1.00), so it cannot come from the extra servers answering more reads. The leader's share of the sessions explains it better: sessions are spread over all servers, so the leader serves a third of them with three servers and a fifth with five, and client connections take 0.4 of the leader's two cores (Section 6.3). Fewer sessions leave the leader more CPU for coordination, which matters only when it is saturated. The log disk (F) has no main effect under either error and appears only in a small E×F interaction (0.5%).

Write tail latency. Load decides the write p99: 160 clients multiply it by 11.7 (94% of the variation), from 2.7 to 31.7 ms (geometric means). With sixteen times more clients and less than twice the throughput, each request finds a longer queue at the leader. The other effects change it by at most a third: payload (×1.32), C×D (without `fsync` the p99 at 10 clients falls from 3.4 to 2.2 ms and at 160 clients stays at 31–33 ms), write ratio (×1.21), `forceSync` (×0.82) and five servers (×1.10, significant only in the whole-plot test). Reads stay fast: their median is 0.27 ms at 10 clients and 0.68 ms at 160.

Pooled against whole-plot error. The whole-plot intervals for A, D and F are narrower than the pooled ones. Had rebuilding the servers added variance, they would be wider, so rebuilds added none that we can detect. With only 7 degrees of freedom the whole-plot error is itself uncertain, so an effect significant in only one test is treated as borderline.

Hypotheses. **H1** holds only at light load: the main effect of D on throughput (×1.07) is borderline and is dwarfed by C×D, which changes its sign; D lowers the write p99 by 18%. **H2** is rejected (B×D, ×1.04, not significant). **H3** is rejected: neither F nor D×F is detectable; since the two disks turned out to have almost the same latency (Section 6.3), this says more about our levels than about ZooKeeper. **H4** is rejected for throughput (five servers are faster) and holds in part for the p99, which five servers raise with 10% writes (7.4 to 9.6 ms)

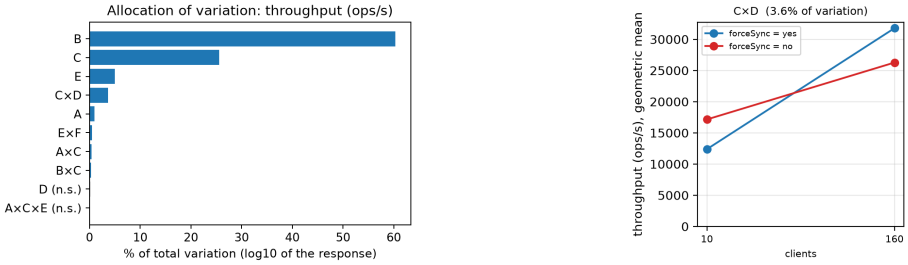


Fig. 2. Left: share of the variation of $\log_{10}$ throughput per effect (ten largest). Right: clients × `forceSync`.

and not with 50% (A×B, ×0.85). **H5** is rejected for throughput, where the write ratio matters more, and holds for the write p99.

6.2 USL curve

We measured one configuration (3 servers, 30% writes, 128 B) at 13 client counts from 1 to 160, with three replicates per level in random order. In the notation of the course,

$$X(N) = \frac{\lambda N}{1 + \sigma(N-1) + \kappa N(N-1)}, \tag{1}$$

where λ is the throughput of one client, σ the contention (serial) coefficient and κ the crosstalk (coherency) coefficient. The fit minimizes relative error, so that 3% counts the same at 600 and at 6,000 ops/s. The intervals come from a bootstrap over the replicates. We compare the full model with the contention-only (Amdahl) model by AIC and an F-test, and validate it by fitting $N \le 64$ and predicting the three highest loads.

Throughput rises from 2,165 ops/s with one client to 46,436 ops/s with 160 (Fig. 3); the client stayed below 28% CPU. The fit gives $\lambda = 2{,}082$ ops/s (95% interval 1,996–2,169) and $\sigma = 0.0385$ (0.0345–0.0414). The crosstalk term is not supported: the Amdahl model has the lower AIC (-241.4 against -239.5), the F-test gives $p = 0.72$, and 37% of the bootstrap fits put κ at zero. We therefore use the Amdahl fit ($\lambda = 2{,}088$, $\sigma = 0.039$), which shows no retrograde region within 160 clients at 30% writes. It bounds throughput by $\lambda/\sigma \approx 53{,}500$ ops/s; at 160 clients the system delivers 87% of that. The efficiency $X(N)/(N\lambda)$ falls to 0.74 at 10 clients and 0.13 at 160. Fitted on $N \le 64$ only, the model predicts the throughput at 96, 128 and 160 clients within 1.9–5.1%. With 50% writes the picture may differ: the factorial runs at 160 clients, 3 servers and 128 B reached 19–21,000 ops/s, less than the 27–29,000 of the probe runs at 80 clients with the same configuration (Section 6.3). The runs come from different sessions, so this only hints at a retrograde region for write-heavy loads.

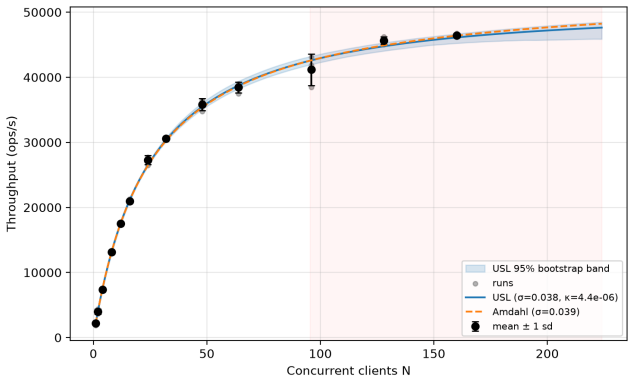


Fig. 3. Throughput against clients: runs, mean ± one standard deviation, the USL fit with its 95% bootstrap band and the Amdahl model. The shaded region was held out for validation.

Response time. Combining the Amdahl fit with Little's law for a closed loop without think time gives

$$R(N) = \frac{N}{X(N)} = \frac{1 + \sigma(N-1)}{\lambda}, \tag{2}$$

a floor of $1/\lambda = 0.48$ ms plus $\sigma/\lambda = 19\,\mu$s for every added client: once the system saturates, each new client only adds waiting time. It agrees with the measured mean N/X within 3.7% at every level (Appendix B), as the throughput fit implies, and lies between the measured median and 95th percentile. At 160 clients reads had a median of 1.5 ms and a p99 of 7.2 ms, writes 6.3 and 13.6 ms.

Per-stage times. Table 5 gives ZooKeeper's timers on the leader at four of the 13 levels (all levels: Fig. 5). The intervals overlap and do not add up: the time to quorum acknowledgment starts when the proposal is created and includes the leader's own log sync and the followers' round trip. From 1 to 160 clients the server-side write latency grows from 0.87 to 5.57 ms. The time to quorum acknowledgment grows most in absolute terms (+2.07 ms), followed by the queue in front of request preparation (+1.35 ms, most of it beyond 64 clients) and the time from commit to applied (+1.11 ms). The log-sync queue levels off at about 0.7 ms, because the batch grows from 1 to 17.6 requests per `fsync`. The `fsync` timer reads only 0.06 ms at $N = 1$ because it truncates each call to whole milliseconds: at least 94% of the calls at that load took less than 1 ms, and only eBPF along the curve could resolve them. Reads show the same hot spot. A read served by the leader takes 2.79 ms at 160 clients against 0.97 ms averaged over all servers, and 1.48 ms of it is spent waiting in the leader's commit stage.

Table 5. Leader per-stage times (ms, mean of the replicates) and requests per `fsync`; the write latency is pooled over all servers. 1 ms timer resolution.

Leader, mean per run	$N = 1$	$N = 16$	$N = 64$	$N = 160$
prep queue (zxid) (ms)	0.02	0.05	0.35	1.37
sync queue (ms)	0.03	0.43	0.66	0.76
sync queue + flush (ms)	0.55	1.08	1.86	2.14
fsync (ms)	0.06	0.22	0.69	0.75
until quorum ACK (ms)	0.72	1.34	2.26	2.79
commit to applied (ms)	0.02	0.08	0.50	1.13
server-side write latency (ms)	0.87	1.61	3.26	5.57
server-side read latency (leader) (ms)	0.05	0.12	0.90	2.79
requests per fsync	1.0	4.1	13.0	17.6

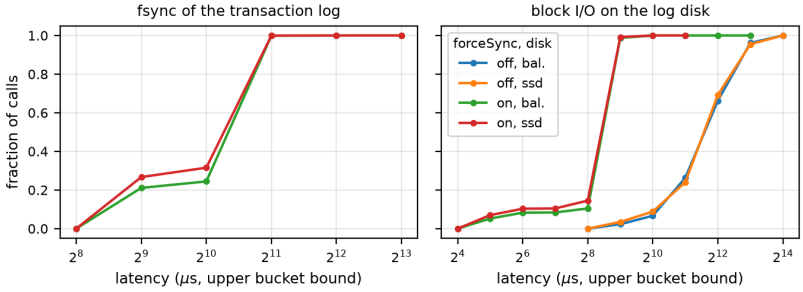


Fig. 4. CDFs of the `fsync` latency and of the log disk's block-I/O latency on the leader, from the $\log_2$ histograms of the probes (one point per bucket).

6.3 eBPF probes

Four probes are our own bpftrace programs [8] and one is bcc's `runqlat`. They measure the latency of ZooKeeper's `fsync`/`fdatasync` calls (syscall tracepoints); the block-I/O latency per disk, from the block tracepoints (bcc's `biolatency` uses a kprobe that kernel 6.1 cannot trace); the run-queue latency of the JVM's threads; TCP retransmissions; and the CPU per thread (a 99 Hz `profile` probe keyed by thread name; ZooKeeper names the thread of each stage). They ran on the leader in separate runs (3 servers, 80 clients, 50% writes) at both levels of `forceSync` and log disk. Each probed run follows a discarded warm-up run and sits between two identical unprobed runs (Fig. 4, Table 6).

The `fsync` and the disk. With `forceSync=no` the probe sees no `fsync`, which confirms that factor D was applied. With it, an `fsync` takes 1.0 ms on average on both disks, and its CDF has two steps: 21–27% of the calls take 256–512 μs and 68–75% take 1–2 ms. The block-I/O CDFs of the two disks almost coincide, with 85–88% of the requests in the 256–512 μs bucket, so neither disk is faster, which agrees with F having no effect. There are about 2.7 block requests per `fsync`. An ext4 `fsync` that must also commit the journal waits for several device round trips in sequence (data, journal, cache flush); that would explain both the

Table 6. The probed runs: throughput, ZooKeeper's counters and the leader's CPU per thread.

`forceSync`, log disk	off, bal.	off, ssd	on, bal.	on, ssd
ops/s with probes	29276	28739	26622	26674
ops/s without (mean of 2)	28795	24992	28731	27301
fsync calls (eBPF)	0	0	21393	23120
fsync mean, eBPF (μs)	–	–	1064	986
fsync mean, ZooKeeper (ms)	–	–	0.77	0.70
requests per flush	1.9	1.8	17.4	16.0
to quorum ACK (ms)	1.14	1.16	2.56	2.52
CPU of the leader (cores of 2)				
replication to followers	0.67	0.67	0.55	0.59
client connections (I/O)	0.40	0.38	0.40	0.39
request preparation	0.20	0.21	0.25	0.25
commit	0.19	0.18	0.17	0.16
log sync	0.10	0.09	0.10	0.11
commit workers	0.06	0.05	0.06	0.06
whole JVM	1.73	1.70	1.61	1.62

1–2 ms step and why an `fsync` costs more than a single device request, although we have not traced the journal. Without `fsync` the log disk sees only about 390 requests in 45 s, mostly of 1–8 ms: background write-back of dirty pages, merged into large requests, which no client request waits for.

CPU, scheduler and network. The leader's JVM uses 1.6–1.7 of its two vCPUs. The largest shares are replication to the followers (0.55–0.67 cores) and client connections (0.4); request preparation uses 0.20–0.25 and the log-sync thread 0.1. Garbage collection uses under 0.003 cores. The run-queue latency has a median of 15 μs and a p99 below 0.5 ms, and there are no TCP retransmissions.

Noise in the probe runs. The two unprobed runs around each probed run differed by 6–43%, and the apparent probe overhead ranges from −15% to +7%. That noise hides any overhead below about 15% and the effect of `fsync` at this load (with probes, runs without it are 9% faster; without probes, 4% slower), so we use these runs only for call counts, latency distributions and CPU shares.

7 Discussion: What Limits Scalability

Contention (σ). Table 7 maps the USL terms to the pipeline. The disk is not the bottleneck: an `fsync` costs about 1 ms on either disk, group commit spreads it over up to 17 requests at high load, and neither D nor F raises throughput at saturation. No single thread is either, since none comes close to a full core. The utilization law points to the leader's CPU as a whole. In the Amdahl model the bottleneck limits throughput to $\lambda/\sigma \approx 53{,}500$ ops/s, so its demand is $D = \sigma/\lambda \approx 18.7\,\mu$s per operation, or 37 μs of CPU per operation if the bottleneck is the leader's two cores ($U = XD/2$). In the probe runs the leader's JVM used 1.6–1.7 cores at 26–29,000 ops/s, that is 59–61 μs per operation at 50% writes, a utilization of 0.80–0.86. Splitting those 60 μs with the cost ratio $w \approx 12r$ gives

Table 7. USL terms and number of servers: candidate stages and evidence.

Term	Candidate stages	Evidence
σ	The leader's CPU: replication, client connections, ordering, log sync	$\sigma = 0.039$; demand 19 μs/op, close to the CPU estimate; JVM at 1.6–1.7 of 2 cores; no thread near a full core; D and F do not help at saturation
κ	Interference between pairs of clients (sessions, GC, commit waits)	Not supported at 30% writes ($p = 0.72$); GC under 0.003 cores; possible at 50% writes
λ	One request alone: round trips, serialization, payload, `fsync`	0.48 ms per request; 4 KiB payloads $\times$0.74; quorum ACK 0.72 ms at $N = 1$
Servers (A)	More followers per proposal vs. fewer sessions on the leader	$\times$1.14, mostly at 160 clients; same at both write ratios; no retransmissions

$r \approx 9\,\mu$s and $w \approx 111\,\mu$s, so about $40\,\mu$s per operation at the 30% writes of the curve. That puts the CPU bound at $2/40\,\mu\text{s} \approx 50{,}000$ ops/s, within 7% of the Amdahl bound. The comparison rests on two assumptions: the cost ratio measured for the whole system also holds for the leader, and the profile at 80 clients holds at 160. Short run-queue waits do not contradict it: threads get a CPU quickly when they wake, but every write adds work to the same two cores, and requests wait for it in the queues of request preparation and of the commit stage, which grow fastest at high load and make reads on the leader three times slower than average. Factor A agrees: moving sessions off the leader helps only at saturation.

Mitigations. The useful changes relieve the leader's CPU: a larger leader machine, a leader that accepts no clients (`leaderServes=no`) [6], observers for reads, and multi-operations, which spread the replication work as group commit spreads the `fsync`. A faster log disk would not help, and turning `fsync` off costs throughput at high load as well as durability.

8 Conclusion

The workload decides ZooKeeper's throughput more than the configuration: the write share explains 60% of the variation of $\log_{10}$ throughput and the load 26%, the log disk nothing, and `fsync` matters only through its interaction with load. The load explains 94% of the write tail latency. The curve at 30% writes needs only contention ($\sigma = 0.039$), and the utilization law and the CPU profile agree within 7% with the leader's two vCPUs being the contended resource. Without validating the load generator, the Stage I artifact would have been reported as a property of ZooKeeper.

Limitations. One zone, shared hosts and 2-vCPU servers; two disk types that a prior `fio` test would have shown to be alike; one USL curve, with probes at one point of another configuration; the leader's share of sessions not recorded; no creates, watches or multi-operations. Next: curves at 0% and 50% writes, probed along the way.

References

1. P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. ZooKeeper: Wait-free coordination for Internet-scale systems. In *USENIX Annual Technical Conference*, 2010.
2. Apache Software Foundation. ZooKeeper: A Distributed Coordination Service for Distributed Applications. ZooKeeper 3.9.3 documentation. `https://zookeeper.apache.org/doc/r3.9.3/zookeeperOver.html`.
3. F. P. Junqueira, B. C. Reed, and M. Serafini. Zab: High-performance broadcast for primary-backup systems. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2011.
4. Distributed Systems Conference. Applying The Universal Scalability Law to Distributed Systems by Dr. Neil J. Gunther. DSConf 2019, March 16, 2019. `https://www.youtube.com/watch?v=cuieNj3NrZ8`.
5. N. J. Gunther. *Guerrilla Capacity Planning*. Springer, 2007.
6. Apache Software Foundation. ZooKeeper Administrator's Guide. ZooKeeper 3.9.3 documentation. `https://zookeeper.apache.org/doc/r3.9.3/zookeeperAdmin.html`.
7. R. Jain. *The Art of Computer Systems Performance Analysis*. Wiley, 1991.
8. B. Gregg. *BPF Performance Tools*. Addison-Wesley, 2019.
9. B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears. Benchmarking cloud serving systems with YCSB. In *ACM Symposium on Cloud Computing (SoCC)*, 2010.
10. A. Ledenev. Pumba: chaos testing and network emulation for Docker containers. `https://github.com/alexei-led/pumba`.

A Stage I experiments not repeated in Stage II

These runs were made on the Stage I laptop with the servers in Docker containers, one run per cell and no confidence intervals. They come from a different testbed and are kept only as context.

YCSB. We ran YCSB [9] with its ZooKeeper binding on 3 servers (Table 8). At 16 threads the read-heavy workloads B and C reach about 6.7 times the throughput of the 50/50 workload A, and in A an update takes 3–4.5 times as long as a read on average (3,713 against 1,078 μs). B and C peak at 16 threads. Workload A jumps from 1,723 to 5,964 ops/s between 16 and 32 threads; with a single run per cell we cannot tell whether this is a real effect or noise, warm-up or garbage collection.

Table 8. YCSB throughput (ops/s), Stage I laptop, 3 servers, one run per cell.

Workload	1 thread	4	16	32
A (50/50 read/update)	790	1316	1723	5964
B (95/5)	3544	6927	11436	8080
C (100% read)	4538	9074	11913	8900

Leader failover. During a continuous benchmark on 3 servers we found the leader with `stat` and killed its container. The two survivors kept a majority and continued, and ZooKeeper logged an election of 211 ms (216 and 235 ms in a repetition). This is ZooKeeper's measurement of the election only, not the outage seen by clients.

Network delay. We added 20±2 ms to the egress of every server container with Pumba [10] (which drives `tc netem` inside each container's network namespace) and repeated the sweep with the quiet client (Table 9). Throughput per client stays at about 21.5 ops/s, one operation per 46.5 ms, at every load: the clients wait on the delay and the servers never saturate within 80 clients. The median read took 22.7 ms (one delayed hop) and the median write 106 ms (about five: forward, proposal, acknowledgment, commit, reply), as Fig. 1 predicts for a client attached to a follower.

Table 9. Throughput (ops/s) with and without a 20 ms egress delay, Stage I laptop, 3 servers, one run per cell.

Clients	1	5	10	20	40	80
No delay	2386	7016	11680	14832	16238	16255
20 ms delay	20.6	108.9	216.7	434.6	865.5	1709

B Response time model

Table 10. Mean response time from the Amdahl fit and Little's law, against measured latencies (all operations).

Clients N	1	8	32	64	128	160
$R(N)$ predicted (ms)	0.48	0.61	1.06	1.66	2.85	3.45
N/X measured (ms)	0.46	0.61	1.05	1.66	2.81	3.45
median (ms)	0.22	0.28	0.49	0.87	1.64	2.42
95th perc. (ms)	1.15	1.79	2.95	4.48	7.50	8.75

C Per-stage times at every load level

D Model checks

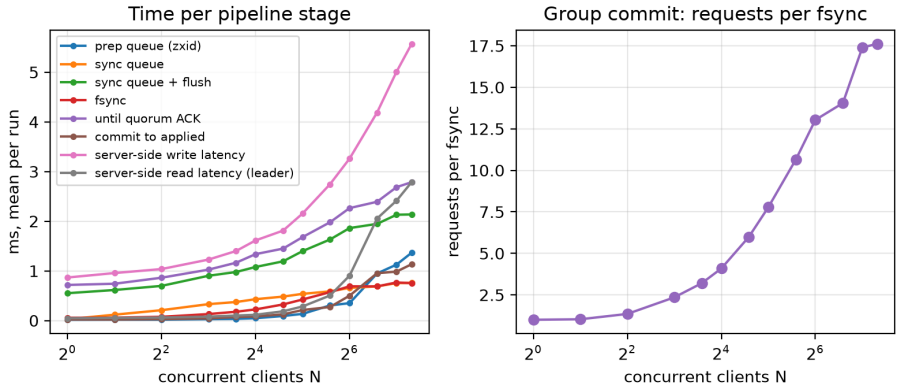


Fig. 5. Leader per-stage times (left) and requests per `fsync` (right) at every level of the USL curve.

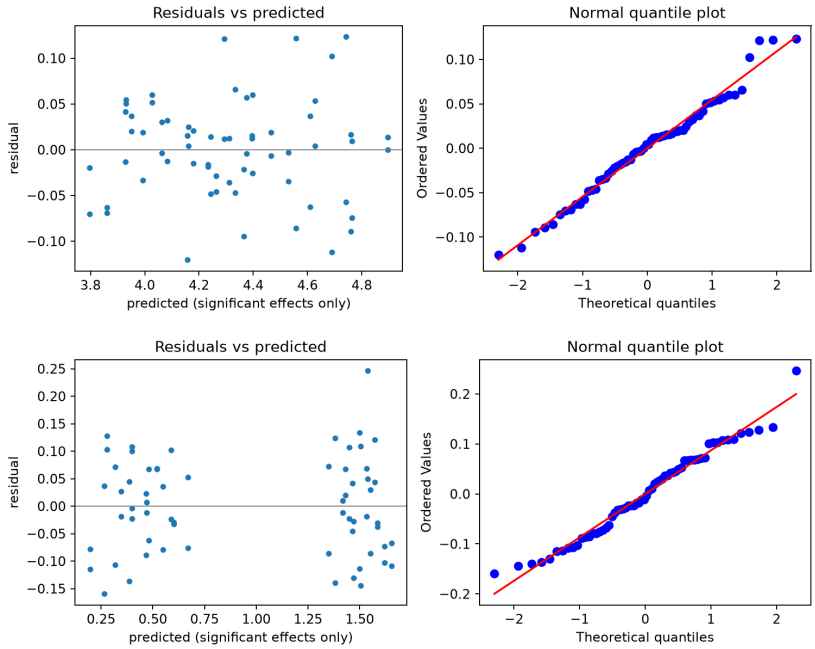


Fig. 6. Residuals of the $\log_{10}$ effects model against the predicted value (left) and normal quantile plots (right): throughput (top) and write p99 (bottom).